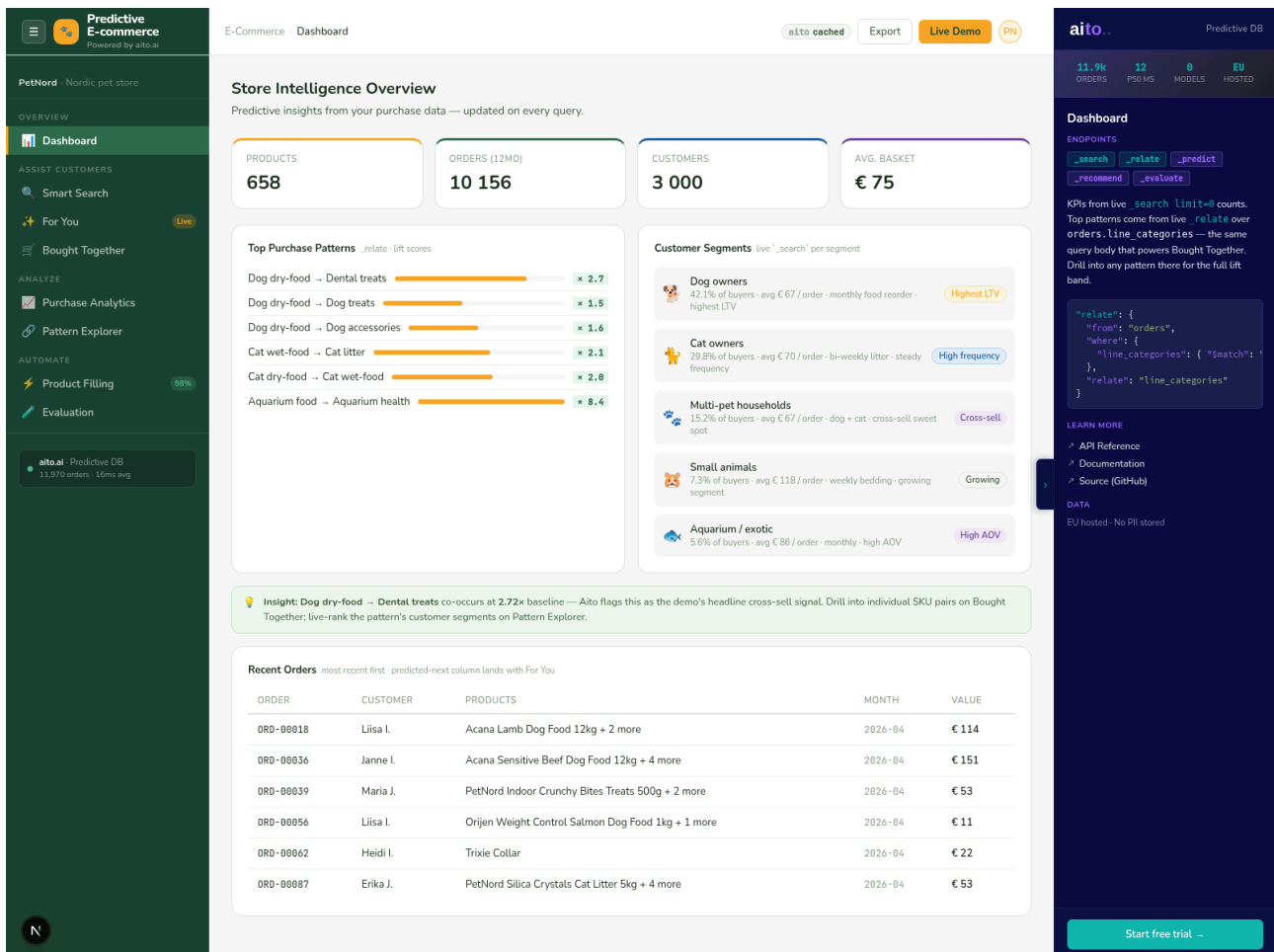


The Shop That Learns

From purchase history. Without training. Without rules.

8 production-ready e-commerce features built on a single predictive database. Predictive search · personalised recommendations · co-purchase intelligence · catalog enrichment · evaluation — all from one query API.



The Challenge

E-commerce predictions traditionally mean MLOps. A trained recommendation model that drifts the week after launch. A search re-ranker that needs a GPU pipeline. A return-risk classifier that nobody can explain when it flags a regular customer. Each model starts useful, drifts as the catalog changes, and rots when nobody owns the retraining schedule.



Models drift in days

Recommendations trained on October data feel stale in November when the seasonal catalog rotates. Nobody's monitoring the precision/recall curves.



Catalog data is messy

Half the SKUs have weight, half don't. New supplier feeds arrive with three of the five attributes missing. Manual enrichment costs hours per launch.



Predictions are unauditable

Black-box ranker returned a cat product to a dog owner. Why? The merchandiser has no recourse beyond "the model said so".

The Solution

Aito is a predictive database. Load your purchase history; query for predictions, recommendations, and statistics through SQL-like calls. No model training. No retraining schedule. No MLOps. Every prediction comes with a \$why decomposition — $\text{base rate} \times \text{pattern lift} \times \text{pattern lift} = \text{final probability}$ — that merchandisers can read and merchandisers can defend.



Zero training

Upload orders, query immediately. The data is the model; every new transaction improves predictions automatically.



Explainable by design

Every ``_predict`` returns the multiplicative chain that produced it. No black box. Click the ? to see why.



Honest when uncertain

``_evaluate`` reports accuracy AND accuracy gain. A 96 % model with +0 pp gain is flagged as honest failure, not shipped as success.

Smart Search — The Rank Flip

The most quotable moment in the demo: type food as a generic query. The standard column on the left ranks by token match — dog food at the top because the catalog is dog-heavy. Switch the customer-context pill from Saara (large-breed dog) to Maija (cat owner) and the right column flips entirely. Every cat-food SKU appears with a gold ★ chip — those products weren't in the standard top-10 at all.

Standard search
Plain token match on the product name.

Rank	Product	Price
1.	Royal Canin Chicken Dog Food 1kg (dog)	€ 6
2.	Royal Canin Chicken Dog Food 2kg (dog)	€ 14
3.	Royal Canin Large Breed Chicken Dog Food 12kg (dog)	€ 72
4.	Royal Canin Sensitive Chicken Dog Food 1kg (dog)	€ 7
5.	Royal Canin Sensitive Lamb Dog Food 15kg (dog)	€ 98
6.	Royal Canin Grain Free Lamb Dog Food 7kg (dog)	€ 49
7.	Royal Canin Lamb Dog Food 2kg (dog)	€ 13
8.	Royal Canin Puppy Lamb Dog Food 2kg (dog)	€ 12
9.	Royal Canin Salmon Dog Food 4kg (dog)	€ 27
10.	Royal Canin Salmon Dog Food 1kg (dog)	€ 7

Predictive search
Re-ranked for Maija — cat owner — same query, customer-context bias.

Rank	Product	Price
1.	Acana Turkey Cat Food 12kg (cat)	€ 73
2.	Whiskas Indoor Whitefish Cat Food 2kg (cat)	€ 12
3.	Orijen Senior Tuna Cat Food 1kg (cat)	€ 6
4.	Whiskas Weight Control Salmon Cat Food 12kg (cat)	€ 72
5.	Acana Grain Free Tuna Cat Food 7kg (cat)	€ 50
6.	Felix Salmon Cat Food 85g (cat)	€ 1
7.	Royal Canin Sensitive Whitefish Cat Food 12kg (cat)	€ 82
8.	Hill's Science Plan Grain Free Whitefish Cat Food 1kg (cat)	€ 6
9.	Royal Canin Sensitive Chicken Cat Food 4kg (cat)	€ 28
10.	Sheba Liver Cat Food 85g (cat)	€ 1

Smart Search
ENDPOINTS
_search _recommend _predict
_relate _evaluate
Re-ranks search results by combining free-text token matching (\$match on the name Text column) with customer-context biasing. Customers see products they're likely to buy, not just products whose names contain the query.

```
{
  "from": "order_lines",
  "where": {
    "product_sku.name": {
      "$match": "food"
    }
  },
  "recommend": "product_sku",
  "goal": {
    "customer_segment": "cat_owner"
  },
  "limit": 10
}
```

LEARN MORE
API Reference
Documentation
Source (GitHub)

DATA
EU hosted - No PII stored

Start free trial ...

What's happening underneath:

- *Standard*: `_search` where `{name: {$match: "food"}}` — token relevance only.
- *Predictive*: `_recommend product_sku` from `order_lines` where `{product_sku.name: {$match: "food"}}` goal `{customer_segment: "cat_owner"}` — Aito ranks the same products by $P(\text{this segment AND this product})$ in the live data.
- The goal field is the only thing that changes between Maija and Saara. The query body is identical otherwise — and the panel shows it on every switch.

A live demo viewer sees one query, three personas, three completely different result lists — without a single model retrained.

For You — Three Crisply Different Shoppers

No query string, no name filter — just the customer's segment + pet size as the goal. The grid re-ranks in under 300 ms per persona flip; the Aito panel updates with the actual `_recommend` body each time.

The screenshot displays the Aito Predictive E-commerce interface. On the left is a dark sidebar with navigation links: Overview, Assist Customers (Smart Search), For You (highlighted), Bought Together, Analyze (Purchase Analytics, Pattern Explorer), and Automate (Product Filling, Evaluation). The main content area is titled 'For You' and shows personalized picks for a selected customer. It includes three tabs for different customer personas: 'Maija — cat owner', 'Olli — multi-pet (small dog)', and 'Saara — large breed dog owner'. Below the tabs is a grid of product recommendations, currently showing a loading state. The right sidebar contains Aito statistics (11.9k orders, 12 p50 ms, 0 models, EU hosted) and a section for endpoints (_recommend, _predict, _relate, _evaluate). It also displays a JSON snippet for a recommendation query and a 'Start free trial' button.

Three personas in one query shape:

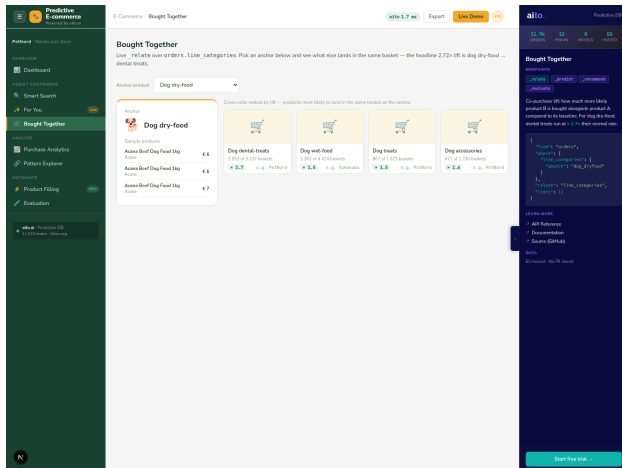
Maija (cat owner) — cat litter, cat dry-food, cat wet-food. Cat consumables top-to-bottom.
Olli (multi-pet, small dog) — dog accessories, grooming, health, toys. Non-food because his segment is the cross-sell shopper.
Saara (large-breed dog) — Acana / Eukanuba / Hill's large-breed kibble. Brand mix swings on `customer_pet_size`.

The split between `where` (filters rows) and `goal` (ranks them) is the load-bearing trick — and Aito's gotcha the demo records: a single multi-field `goal`: `{segment, pet_size}` collapses onto whichever field dominates. Splitting the constraint gives every persona a sharp result.

Co-Purchase Intelligence

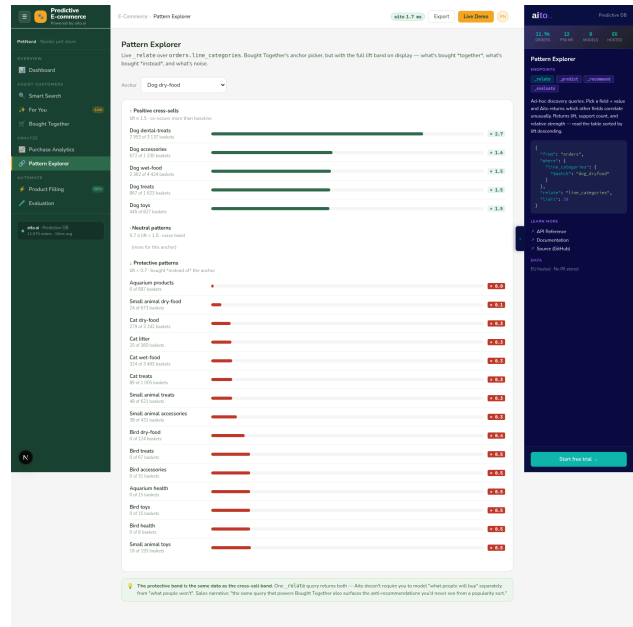
Bought Together

Anchor a product, see four cross-sell tiles with live lift scores. Dog dry-food → dental treats at $2.72\times$ baseline. The Aito panel shows the live `_relate` body that drove it.



Pattern Explorer

Same `_relate` query, full lift band on display. The positive band is Bought Together's cross-sells. The protective band ("dog dry-food → cat wet-food $\times 0.27$ ") is the **anti**-recommendation Aito gives you for free.



The Aito API doesn't expose order-level co-occurrence directly — Aito's `_relate` operates within-row. A denormalised orders.line_categories Text column on the schema (each order's `<pet>_<category>` tokens space-separated) makes the query work in one hop. One small column, two views, one of the five demo moments — documented openly in the ADR.

Catalog Enrichment — Five Fields in Parallel

Five `_predict` calls in parallel: `pet_type`, `category`, `weight_kg`, `dietary`, `tax_class`. All from `name` + `brand`. Five round-trips, 500 ms end-to-end. Every prediction returns a confidence chip and a `$why` decomposition.

The screenshot displays the Predictive E-commerce interface. On the left is a dark green sidebar with navigation links: Overview, Assist Customers (Smart Search, For You, Bought Together), Analyze (Purchase Analytics, Pattern Explorer), and Automate (Product Filling, Evaluation). The main content area is titled 'Product Filling' and shows an 'Incomplete product' dropdown set to 'Hill's Science Plan Sensitive Turkey Dog Food 2kg'. Below this, two panels compare the input data with Aito's predictions. The left panel shows the input: Pet type (dog, stored), Category (dry-food, stored), Weight (kg) (null), Dietary (null), and Tax class (food-reduced). The right panel shows Aito's predictions: Pet type (dog, 98%), Category (dry-food, 87%), Weight (kg) (2, 98%), Dietary (sensitive, 95%), and Tax class (food-reduced, 98%). A right sidebar shows Aito's Predictive DB status (11.9k orders, 12 p90 ms, 0 models, EU hosted) and a JSON query for the product filling process. At the bottom right is a 'Start free trial' button.

Field	Input	Aito Prediction	Confidence
Pet type	dog (stored)	dog	98%
Category	dry-food (stored)	dry-food	87%
Weight (kg)	— null —	2	98%
Dietary	— null —	sensitive	95%
Tax class	food-reduced	food-reduced	98%

What this catches:

- **Missing weight** — “2kg” in the name tokenises to `weight_kg` = 2.0 with 98 % confidence
- **Missing dietary** — “Sensitive” tokenises to `dietary` = sensitive with 95 %
- **Wrong tax class** — food-reduced predicted with 98 % from category × brand co-occurrence
- **Two locked fields** (`pet_type`, `category`) — already stored; Aito predicts them anyway and the UI tags them stored for honesty

The Text type on `products.name` is load-bearing — Aito tokenises the column and uses individual words (“Sensitive”, “2kg”, “Dog Food”) as features. A schema with `name: String` would collapse the moment to exact-name lookup.

Evaluation – Honest by Design

Four `_evaluate` models. The threshold is `accuracy_gain ≥ +10 pp`. Three pass. One deliberately fails — `return-risk` gains **+0.0 pp** over the prior. The `accuracy` column reads 96.5 %; the `gain` column reads 0.0 pp.

The 96 % isn't earned. About 3 % of order lines get returned regardless of features; predicting "won't be returned" for every line is correct 96.5 % of the time and adds nothing the prior didn't already know. That's what +0 pp gain means — and that's the moment most demos hide.



Predictive E-commerce

Powered by aito.ai

E-Commerce

Evaluation

aito 1.8 ms

Export

Live Demo

PH

PetNord

Nordic pet store

OVERVIEW

Dashboard

ASSIST CUSTOMERS

Smart Search

For You

Bought Together

ANALYZE

Purchase Analytics

Pattern Explorer

AUTOMATE

Product Filling

Evaluation

aito.ai Predictive DB

11.970 orders 180ms avg

Evaluation

Hold-out accuracy per prediction model. **Pass** when Aito's accuracy beats the baseline by ≥ 10 pp. **Fail** means Aito is honestly telling you it doesn't know — even when the raw accuracy is high.

Last run: 2026-05-11T11:31:27+00:00 · 35589 ms · 4 parallel "evaluate" calls

	MODEL	FEATURES	ACCURACY	BASELINE	GAIN	N	VERDICT
✓	Pet type from product name predict pet_type from products	name, brand	97.5%	0.0%	+97.5pp	200	PASS
✓	Dietary tag from product attributes predict dietary from products	name, brand, category, pet_type	71.5%	24.5%	+47.0pp	200	PASS
✓	Customer segment from product attributes predict customer_segment from order_lines	product.pet_type, product.category	88.5%	46.5%	+34.0pp	200	PASS
✗	Return risk (deliberate honest-failure case) predict returned from order_lines	product.category, product.pet_type, customer_segment	96.5%	96.5%	+0.0pp	200	FAIL

💡

The failure that earns trust: Return Risk's accuracy looks impressive (~96%) but its gain is zero — Aito learned nothing the prior didn't already know (= 3 % of lines get returned, regardless of features). That's the model honestly telling you "I can't predict this from your current data." Better than a fake high-accuracy score on a production deployment.

aito.ai

Predictive DB

11.9% ORDERS

12 PSR MS

0 MODELS

EU HOSTED

Evaluation

ENDPOINTS

evaluate

predict

recommend

relate

Aito's **evaluate** holds out a test set, runs predictions, and reports accuracy + baseline accuracy + per-case results. Honest failure cases (the return-risk model below) live here — Aito tells you when it doesn't know.

```

{
  "testSource": {
    "from": "order_lines",
    "limit": 200
  },
  "evaluate": {
    "from": "order_lines",
    "where": {
      "product_sku.category": {
        "$get": "product_sku.category"
      },
      "product_sku.pet_type": {
        "$get": "product_sku.pet_type"
      },
      "customer_segment": {
        "$get": "customer_segment"
      }
    },
    "predict": "returned"
  },
  "select": {
    "accuracy",
    "baseAccuracy",
    "n"
  }
}

```

LEARN MORE

API Reference

Documentation

Source (GitHub)

DATA

EU hosted · No PII stored

Start free trial

Why this is the demo's most trust-building moment: every model ships with a held-out evaluation. Aito tells you when its predictions are real signal and when they're a coin flip dressed as accuracy. The return-risk row is the one a CTO would deploy under any naïve metric — and the one Aito refuses to let you ship by labelling it FAIL.

How It Works



1. Connect your data

Catalog + customers + orders + order_lines → Aito instance. JSON; schema declared in `data\_loader.py`. ~53k rows uploads in 50 s.



2. Query for predictions

Five operators cover all 8 use cases:

- `_predict`: catalog fill
- `_recommend`: search + For You
- `_relate`: co-occurrence
- `_search`: KPIs + analytics
- `_evaluate`: honest pass/fail



3. Integrate

REST API. ~30 ms response time. Sub-1 ms on warm-cache. Two-tier rate limit + persistent cache built in. Drop into your storefront or build standalone.

Architecture at a glance

Backend — Python FastAPI · one service module per view (`overview_service`, `search_service`, `recommend_service`, ...)

Frontend — Next.js 16 (App Router) · TypeScript strict · locked Aito side panel on every route

Aito — REST API · `_predict` / `_recommend` / `_relate` / `_search` / `_evaluate`

Cache — Two-layer (in-memory + Aito-backed `prediction_cache`); no-op in `PUBLIC_DEMO=1` mode

Schema — 4 tables; chained-link denormalisation (`order_lines.customer_segment` / `customer_pet_size`, `orders.line_categories`) makes single-hop `_relate` + `_recommend` work cleanly

Public-demo mode — `PUBLIC_DEMO=1` toggles CORS lockdown, two-tier rate limit, memory-only cache, schema 404

Aito gotchas the demo records

The cheatsheet (`docs/aito-cheatsheet.md`) documents seven verified live patterns and seven Aito-API gotchas surfaced while building. Among them: `$match` is required for Text tokenisation (plain "food" returns zero hits), multi-field `goal` doesn't behave as AND, Aito tokenises Text on whitespace AND hyphens, `_evaluate` requires both `testSource` AND `evaluate` blocks. Future demo-builders skip the probe rounds.

What This Demo Doesn't Try to Be

This is a **predictive-database reference for e-commerce**, not a complete storefront. The capabilities the demo deliberately omits — and what production would add — are documented openly. A non-exhaustive list:

Checkout / payments — no Stripe wiring, no cart persistence. The demo stops at recommendation; production wires the predictive surfaces into the existing storefront's cart and checkout.

Per-customer history — Aito's `_recommend` underfits on 3 000- customer × few-orders datasets. Segment-level conditioning is the pattern shown; per-individual personalisation needs an order of magnitude more rows and `aito-shopify`'s impressions schema.

Multi-locale catalogs — single Finnish-flavoured catalog. Production adds locale-keyed product tables and routes the `$match` query against the locale's tokeniser.

Image-based search — predictions read off the product name's Text tokens. Image embedding + similarity is a separate capability Aito doesn't ship; production stacks it alongside.

Return-policy enforcement — return-risk is the deliberate honest- failure case. Production wouldn't ship the model with +0 pp gain; it'd surface the evaluation outcome to the merchandising team and return to feature engineering.

Inventory / supply chain — explicitly out of scope; the ERP demo (`aito-erp-demo`) covers that surface. Two demos, two predictive perimeters.

Owning the gaps is more credible than papering over them. Each row above is a real objection raised by e-commerce CTOs reviewing the demo. Production checklist + scaling guidance lives in the ADRs (`docs/adr/`) — 11 architecture decision records documenting every load-bearing choice.

Ready to Try It?



Read the code

Apache 2.0 on GitHub. 11 ADRs.
~5k lines Python + TS.
Production-quality reference —
fork it for your vertical.



Run it locally

```
./do setup && ./do load-data  
&& ./do dev. Bring your own  
Aito API key (free tier covers the  
PetNord 53k rows comfortably).
```



Talk to us

If your e-commerce roadmap has
predictive search,
recommendations, or catalog
enrichment on it, we should talk.
EU hosted, no PII stored.

Predictive intelligence for your shop — without the ML pipeline

Aito.ai is a predictive database. Upload orders, query for predictions, ship features. The 8 capabilities in this demo are the same query API you'd use in production — `sub-100 ms _predict` calls, full `$why` explanations, and honest `_evaluate` built in.

hello@aito.ai · aito.ai · github.com/AitoDotAI/aito-ecommerce-demo

Aito.ai builds predictive infrastructure for product teams who want statistical intelligence without standing up an ML team. Open-source demos: `aito-demo` · `aito-accounting-demo` · `aito-erp-demo` · `aito-ecommerce-demo`.